ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: Erik William Andreasson**
- **Chinese Name:**
- **Email: eriub@student.chalmers.se**
- **Student ID: X1150052**
- **University: National Tsing Hua University**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

1. There are 594 documents in the category “sci.med”

2. By isolating specific categories before applying the technique we can find patterns, here sets of words, that are frequent in each category, as opposed to the patterns frequent across the whole dataset. With per category patterns we can more easily use them to make conclusions on the type of vocabulary that is frequently used in these categories which in turn may help in differentiating records from each other for downstream classification tasks. If we were to apply this technique to the full dataset without any separation then we would not be able to determine if any given pattern is more common in one category over the other.

AI used for this answer: Yes / No (**No)**

If yes, how:

External references (links):

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer:
Simply dropping 40% of the documents would severely impact the results of our applied algorithms and our analysis of the data. Because, by removing these records we also remove the large amount of perfectly good text data along with it. This large information loss results in fewer training examples, altered category distribution, and potentially removed categories. For example, let’s say there was a systemic error in the data collection and nearly all documents with the categories science, art, and literature were never categorized. Then naively removing all of those records would create a very imbalanced

class distribution which could lead one to believe that these categories are severely underrepresented in the dataset. Furthermore, class imbalance makes downstream tasks like classification much more difficult for these categories.

An objective alternative is to try to estimate the missing label from the record's text, using the labeled documents. If the missing categories are randomly distributed, as opposed to a systemic error, the estimate can be modeled as a probabilistic classification problem where one specific method is applying a semi-supervised learning Naive Bayes Classifier with the Expectation-maximization algorithm. Here we define the category-name as the latent variables of which we calculate the posterior probabilities for. Then we can take the argmax of the posterior probabilities, with a lower bound to take only confident probabilities, over each category as our estimated category. This allows the records with already labeled categories to provide a strong signal of truth to the EM algorithm. Therefore, we can still utilize the large amount of textual content in the dataset despite missing category labels.

AI used for this answer: Yes / No **(Yes)**

If yes, how:

AI was used to discuss concrete applications of estimating the missing **category_name** label and how such a method is improved by existing “truthful” data.

The approach I suggested in my answer is based on the work linked below which attempts to show the same concept but on classification on unseen data. It can similarly be applied to label recovery.

External references (links):

Nigam, K., Mccallum, A.K., Thrun, S. et al. Text Classification from Labeled and Unlabeled Documents using EM. Machine Learning 39, 103–134 (2000). https://doi.org/10.1023/A:1007692713085

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer:

1. No the proportions do not perfectly match the original. There are random slight differences. However, overall they are very close to the same proportions since the categories are already roughly balanced. For example, using numbers from the notebook, comp.graphics goes from 25.9% to 27.8%.

2. Simple random sampling does not ensure that the category proportions are preserved in a sampled dataset. On a highly imbalanced dataset this can make the sampled dataset more imbalanced by misrepresenting minority categories or removing them entirely by chance. For example, a category with 1% distribution of the data is expected to have 10 occurrences in a random sample of 1000 but could have 3, 10, or 0. This can severely impact model training, algorithm results, and analysis of our data.

AI used for this answer: Yes / No (**No)**

If yes, how:

External references (links):

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:

1. The total number of features increases to 35788 from 35482

2. Adding stopwords back in reduces the overall sparsity of the matrix. These words are highly frequent in documents, so their columns have a higher density of non-zero entries than the rest of the matrix. Therefore, overall percentage of zeros drops. Re-running with stop_words=None gave a sparsity drop from 99.7% to 99.5%.

AI used for this answer: Yes / No **(No)**

If yes, how:

External references (links):

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

Threshold	**Number of terms kept (keep == True)**
0.0	35482
0.01	5089
0.05	1491

1. The total size of the vocabulary shrinks as the variance threshold increases.

2. The kinds of words that have a variance close to zero are either the ones that appear in many documents the same amount of times or those that appear in almost no documents. Words of the former type are less likely in this dataset where the text lengths have a high variance. Therefore, it is unlikely any such word exists and would be removed. However, as in most text there are a lot of words that are exceptionally rare with only a few appearances in any document which makes the document frequency for such a word very low. As a result, the variance is low and the filter removes them. Examples for this kind of word are given in the notebook and are usually very specific and uncommon words like "zimmerman" but are also rare typos or terms like “zsoft” or “zool”.

3. Eliminating such features can greatly reduce the dimensionality of the problem which allows methods that were previously too computationally intensive to become viable on the dataset. It is relatively safe to remove features with low variance without greatly reducing accuracy because low variance features do not differentiate records well. If almost every record has the same value for a feature, then knowing the value of that feature for any given record does not provide more information about the record than for any other record. Therefore, I cannot reliably use this feature to make an informed decision on the records. However, it is possible that some rare words are very specific to categories and would be useful to differentiate records and correctly classify a document. Therefore, it is important to set the variance threshold accordingly to safely remove noise while keeping actually unique and informative domain-specific terminology.

AI used for this answer: Yes / No **(No)**

If yes, how:

External references (links):

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

1. Stopwords specifically are already removed from the previous filtering step leaving only generics and domain-specific terminology.

The Pearson list includes words with positive correlation like "thanks", "keywords", "looking" as highest with domain-specific words "vga", "video", and "card" lower down in the list. The former are generic words in headers or emails without any direct connection to computer graphics. However, the latter domain-specific words in the top 10 are directly connected to computer graphics. The terms with negative correlation are also generics like "people" and "writes".

The Spearman list has positive correlation for "graphics", "files", "image", "3d", among others in the top words. All of these have a clear and direct connection to the category. There are still some generic words among the top 10 like "people" and "thanks" but they are much less than in the Pearson list.

2. Pearson correlation measures the covariance normalized by both standard deviations of the observed values directly to find a strictly linear relationship. Spearman is Pearson but applied to the ranks of the observed variables, meaning the values are converted to ordinal values 1-n with ties averaged, to find any monotonic relationship. This means that Pearson can only find linear relationships for correlating variables while Spearman can find correlations for variables based only on their relative order to each other regardless of their actual value. This allows for monotonic non-linear relationships between two variables to have a high Spearman value and a low Pearson value.

3. One specific scenario where Spearman has a high rank and Pearson is low is the case where the relationship is composed of monotonic values with large outliers. For example, out of 300 documents, 100 of them are not labeled comp.graphics (indicator is 0) and they all have zero term counts for a given term X. 100 documents have indicator 1 with each document having count 1 and the final 100 documents have indicator 1 with each document having a count of 50. In the case of Pearson this situation would provide a positive covariance, but the spread in the values with indicator 1 is so high that the standard deviation is also very high, making the total coefficient lower. Spearman does not take into account this spread, instead, the documents with high count and low count are simply ranked as ordinal data which compresses the outlier values into a tighter range

than their magnitude creates. As a result, the data receives a much higher correlation than that for Pearson.

AI used for this answer: Yes / No **(Yes)**

If yes, how:

AI was used to discuss the differences in the mathematical definitions of Spearman and Pearson. Specifically to understand how the Rank value is defined in the case of variable values that tie and the effect of that on the Spearman value.

External references (links):

https://en.wikipedia.org/wiki/Spearman%27s_rank_correlation_coefficient

https://en.wikipedia.org/wiki/Ranking_(statistics)

https://en.wikipedia.org/wiki/Pearson_correlation_coefficient

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html

---

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup:

minSup = 3: 9547 patterns, 1.26 seconds

minSup = 9: 710 patterns, 0.075 seconds

As we reduce the minimum support required for a pattern we find more patterns that frequently occur in the dataset. This also leads to a longer search time and more computation on finding the combinations of words with that support or higher.

Answer (2) — Top-K k vs MaxFPGrowth minSup:

The parameter k specifies directly the number of patterns you wish to find. The Top-K FAE algorithm dynamically finds the support value required to produce the top k most

frequent patterns in the dataset. This fundamentally differs from the minSup parameter since the number of patterns returned depends only on how many transactions each pattern is found in. Specifying a minSup of 3 will give many more patterns than for a value of 9 since many more patterns occur in at least three transactions as opposed to nine. On the other hand, specifying a k =100 will always give you the 100 most frequent patterns, unless of course you have exceptionally little data.

Answer (3) — Maximal vs standard frequent patterns:

They are considered more concise in the terms of the output of the algorithm because the subsets of the pattern are removed from the results. For example, if a maximal frequent pattern of {“mind”, “thought”, “feel”} is found, then the results would not include all the subsets {“mind”, “thought”}, {“mind”}, …., and so on. In this way they are structurally more concise since the maximal pattern includes the other patterns as well without adding them to the result, since any subset of the pattern must have at least the same support or higher. In contrast, the standard frequent patterns would include the maximal pattern as well as all of the subsets since all of them exceed the minimum support.

AI used for this answer: Yes / No **(Yes)**

If yes, how:

I wrote a prompt describing my thoughts and hypothesis for question 2 and 3 while requesting it to point me to further reading on the specific algorithms to learn the precise differences between them. Therefore, providing me with the sources to answer the question.

External references (links):

Z. -H. Deng and G. -D. Fang, "Mining Top-Rank-K Frequent Patterns," 2007 International Conference on Machine Learning and Cybernetics, Hong Kong, China, 2007, pp. 851-856, doi: 10.1109/ICMLC.2007.4370261. https://ieeexplore.ieee.org/document/4370261

Bayardo, R. J., Jr. 1998. Efficiently Mining Long Patterns from Databases. In Proceedings of the 1998 ACM SIGMOD International Conference on Management of Data (SIGMOD ‘98), Seattle, WA, USA, 85-93. https://dl.acm.org/doi/10.1145/276304.276313

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:

1. It is impossible to visually plot any point with more dimensions than three. This is because paper and screens can only display three dimensional axes. Therefore, representing a document in a visually comprehensible way as a vector with thousands of dimensions on a screen is not possible. As a result, we cannot truly represent the distances between points (documents) since we must perform a lossy reduction of the dimensionality first.

2. Dimensionality reduction.

AI used for this answer: Yes / No **(No)**

If yes, how:

External references (links):

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

1. PCA has the most distinct overlap where nearly all documents are pushed into a single point. Both t-SNE/UMAP show more distinct cluster islands for the four categories. However, there is still some intermixing in the middle for both methods. Specifically, t-SNE clusters all of the documents into a single ball where it appears that certain partitions of the ball have distinct groupings for the four categories. UMAP also has more separated clusters for categories, yet the spread of the entire dataset is much less uniform than for t-SNE.

2. PCA works by preserving the components, or directions, that represent the highest variance in the data via a linear projection. The idea is to preserve the spread which retains the information content of the dataset. This relates to previous discussions on how low variance features do not provide information that can differentiate records. The same concept is being applied here. However, the three directions with highest variance do not necessarily differentiate the cluster effectively. Furthermore, PCA is more sensitive to large outliers which push the variance higher, potentially making an insignificant direction the component that is kept. Methods like t-SNE and UMAP work on local non-linear relationships between points which allows them to be more robust against outliers

and more directly preserve local relationships than PCA. This allows them to more easily capture the cluster information in the dataset with reduced dimensionality.

3. The physical distance is difficult to interpret in a t-SNE plot since the objective function does not require relative distances to be kept within the dataset. Instead the objective function focuses on massively penalizing placing two points that are neighbors in the original space as points far apart in the target space. In contrast, placing two points that are not neighbors in the original space closer together in the target space is penalized far less. This pushes the objective function to force neighbors together, creating local clusters, but also potentially place these distinct clusters closer together than they really are in the original space. Furthermore, the spread and density within clusters are roughly equally normalized to the same size, making each clusters apparent size and density a poor representation of the source space.

AI used for this answer: Yes / No **(Yes)**

If yes, how:

I discussed the specifics of how the algorithms **PCA, t-SNE,** and **UMAP** worked using references. This discussion aided me in understanding their differences.

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html

https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html

van der Maaten, L.J.P.; Hinton, G.E. Visualizing High-Dimensional Data Using t-SNE. Journal of Machine Learning Research 9:2579-2605, 2008.

McInnes, L., Healy, J. & Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. arXiv:1802.03426. https://arxiv.org/abs/1802.03426

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1. Four binary columns, one for each category.

2. It can falsely interpret the data as both representing meaningful distances from each other as well as misinterpreting as ordinal data where one category is valued higher than the other. In reality, they are just nominal data representing a single label.

3. It converts the data from the integer encoding to representing each category as a 1 if present and the rest as zeros. So a document with category 0 would have encoding [1, 0, 0, 0]. This prevents algorithms from interpreting the data as ordinal where one category is more valuable than the other since any pair of encoded categories is equidistant.

AI used for this answer: Yes / No **(No)**

If yes, how:

External references (links):
